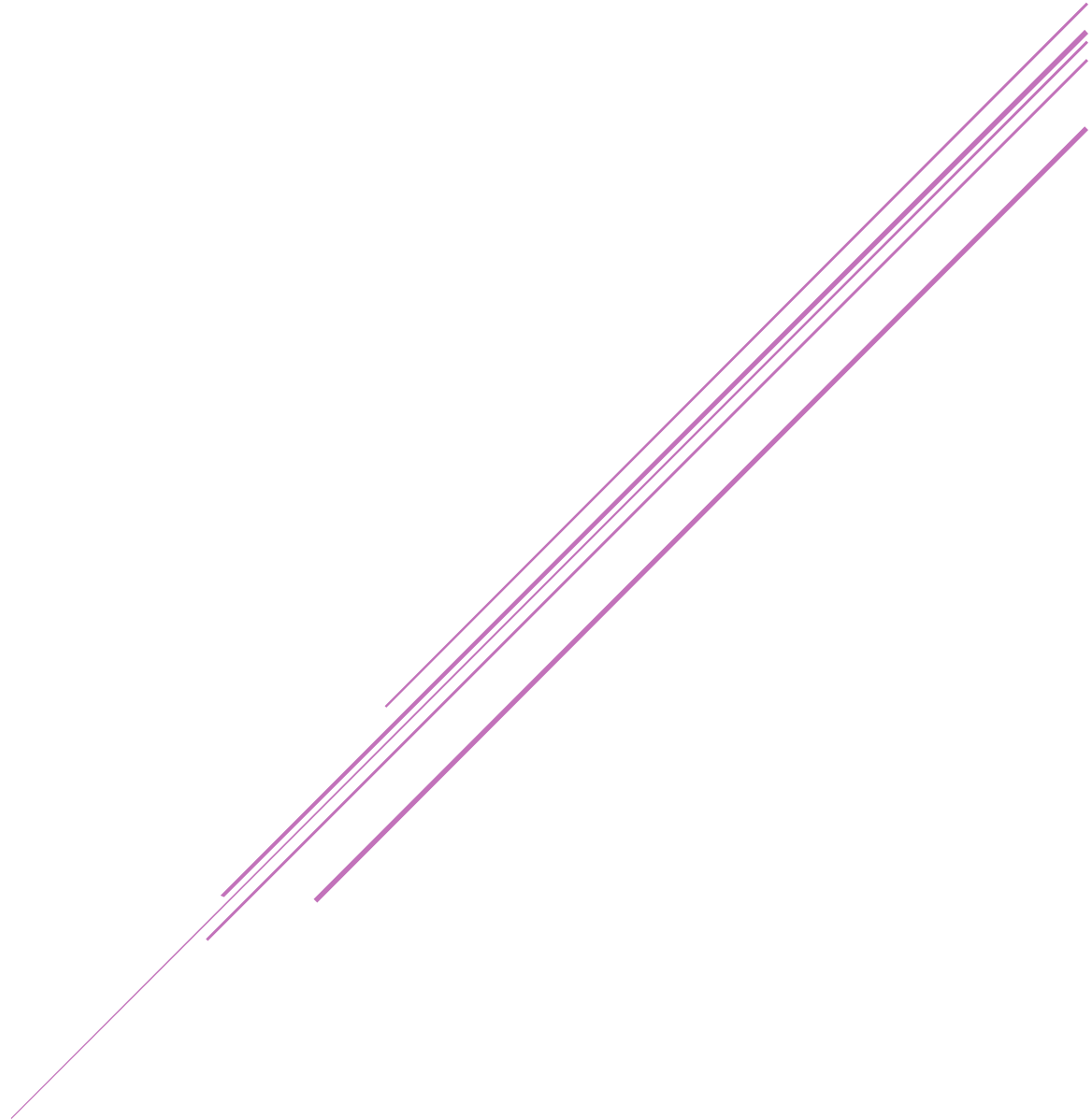


GitHub and Digital Repository Management

SPRINT 3

IN-MEXICO PROGRAM BACKEND

DEVELOPER CERTIFICATION



Contents:

Creation of branch "A" from "Main"	2
1. Creation of docs.txt in branch "A":.....	2
2. PR created for branch "A":.....	4
Creation of branch "B" from "Main"	5
3. Creation of a different docs.txt in branch "B"	5
4. PR created for branch "B"	7
Conflict resolution process	8
Git command used to revert changes	9
Approved PRs and merged branches	10
Notes for feedback:	11

Creation of branch "A" from "Main"

From branch Main we just must write the command "git branch name.branch" in this case the branch will be called "A":

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git branch A
```

Then, switch to the new branch with the command "git checkout branch.name":

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git checkout A
Switched to branch 'A'
```

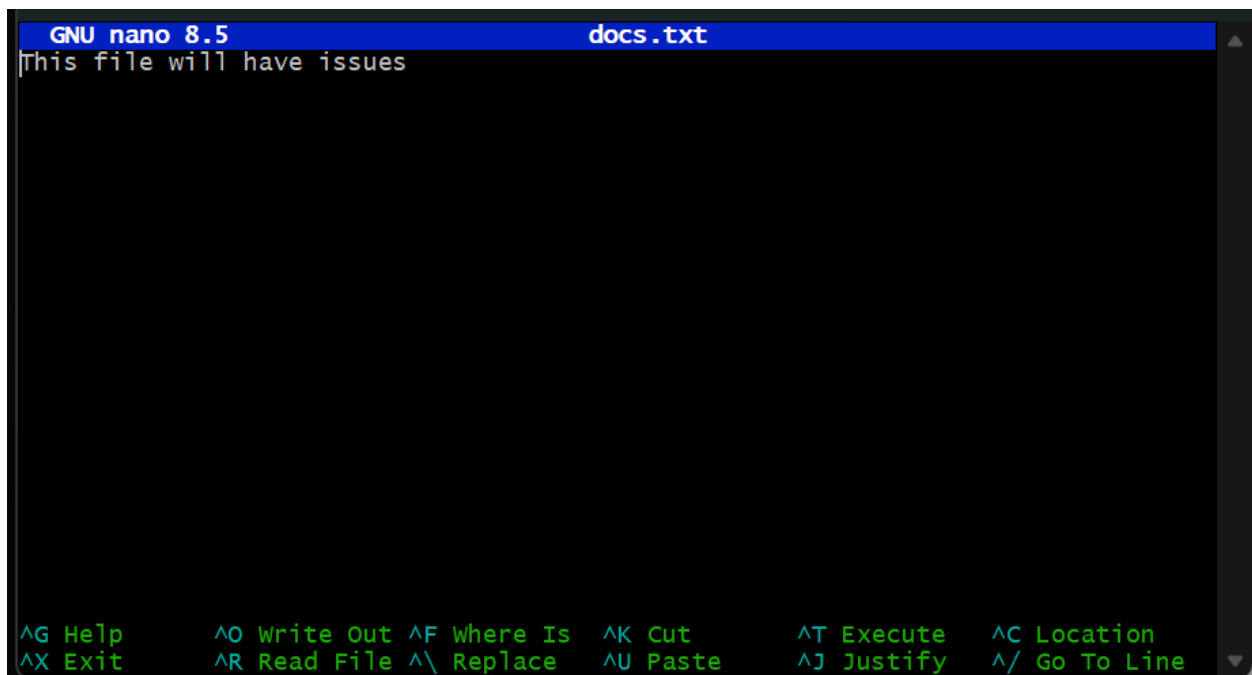
1. Creation of docs.txt in branch "A":

Inside the desired branch, write down the command "touch name_file.txt":

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (A)
$ touch docs.txt
```

Add a few lines with the command "nano name_file.txt":

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (A)
$ nano docs.txt
```



```
GNU nano 8.5 docs.txt
This file will have issues

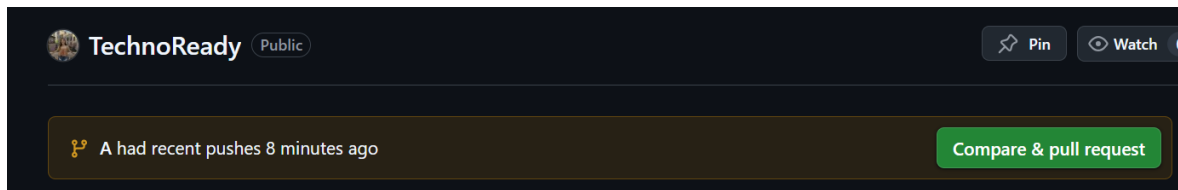
^G Help      ^O Write Out ^F Where Is   ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace    ^U Paste     ^J Justify   ^_ Go To Line
```

Add, commit and push all the changes with the command “git push -u origin A”:

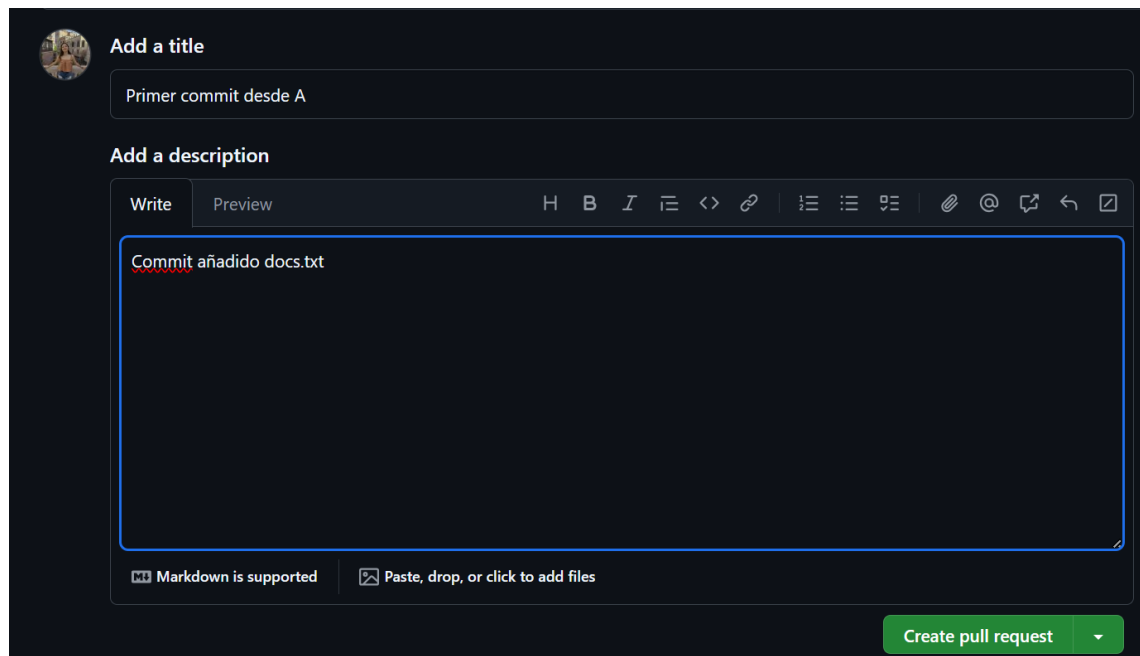
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (A)
$ git push -u origin A
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 579 bytes | 289.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'A' on GitHub by visiting:
remote:   https://github.com/valduque/TechnoReady/pull/new/A
remote:
To https://github.com/valduque/TechnoReady
 * [new branch]      A -> A
branch 'A' set up to track 'origin/A'.
```

2. PR created for branch "A":

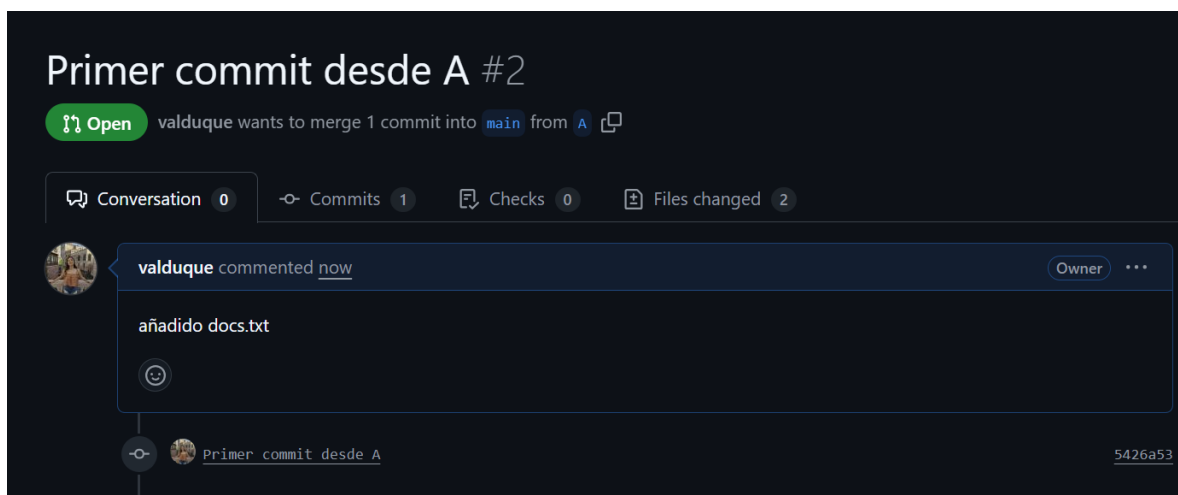
On the project's Github repository page, it will show the new branch and pushes created by it. "Compare and pull request" will be clicked:



Then, the customizable page for the pull request will appear:



Once finished all the details, just press "create pull request":



Creation of branch "B" from "Main":

From branch Main we just must write the command "git branch name.branch" in this case the branch will be called "B":

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git branch B
```

Then, switch to the new branch with the command "git checkout branch.name":

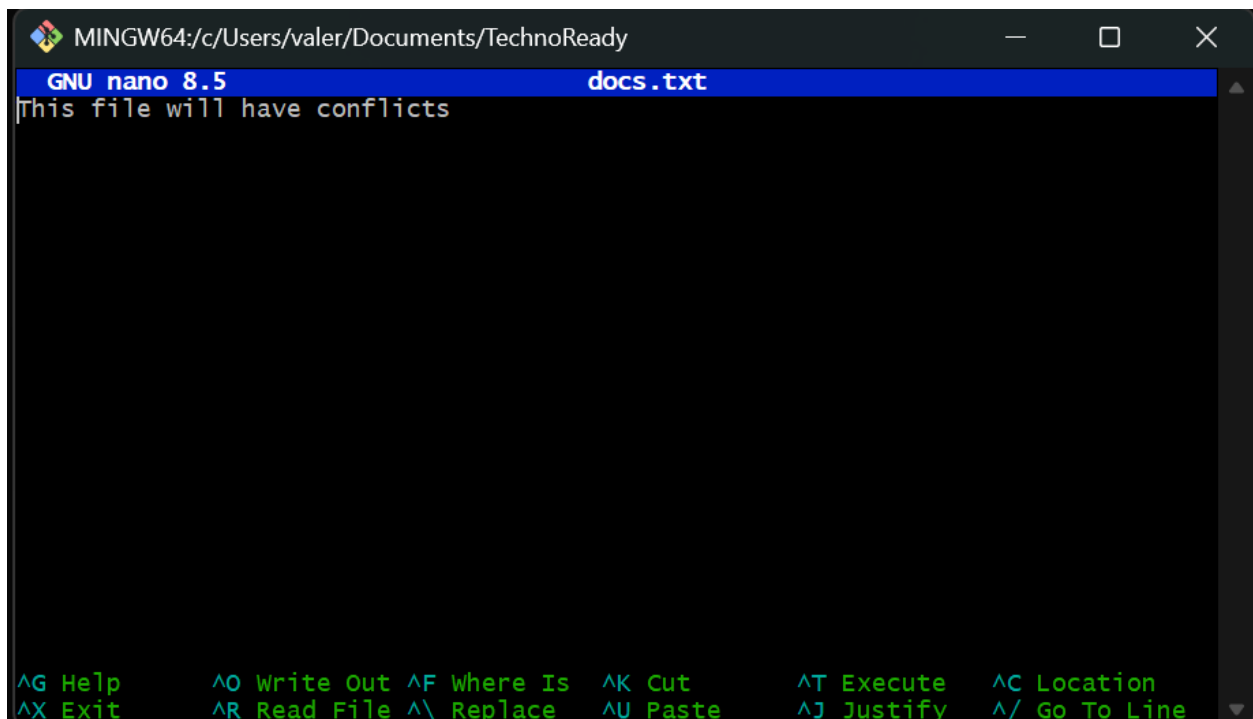
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git checkout B
Switched to branch 'B'
```

3. Creation of a different docs.txt in branch "B":

Inside the desired branch, write down the command "touch name_file.txt":

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ touch docs.txt
```

Add a few lines with the command "nano name_file.txt":



```
MINGW64:/c/Users/valer/Documents/TechnoReady
GNU nano 8.5 docs.txt
This file will have conflicts

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Add, commit and push all the changes with the command "git push -u origin A":

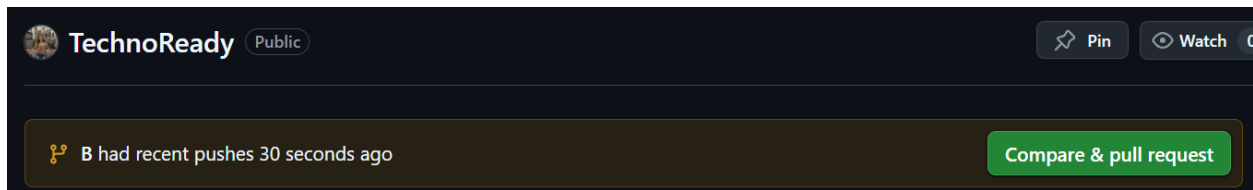
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git add .
warning: in the working copy of 'docs.txt', LF will be replaced by CRLF the next
time Git touches it
```

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git commit -m "Primer commit en B"
[B a91ba89] Primer commit en B
1 file changed, 1 insertion(+)
create mode 100644 docs.txt
```

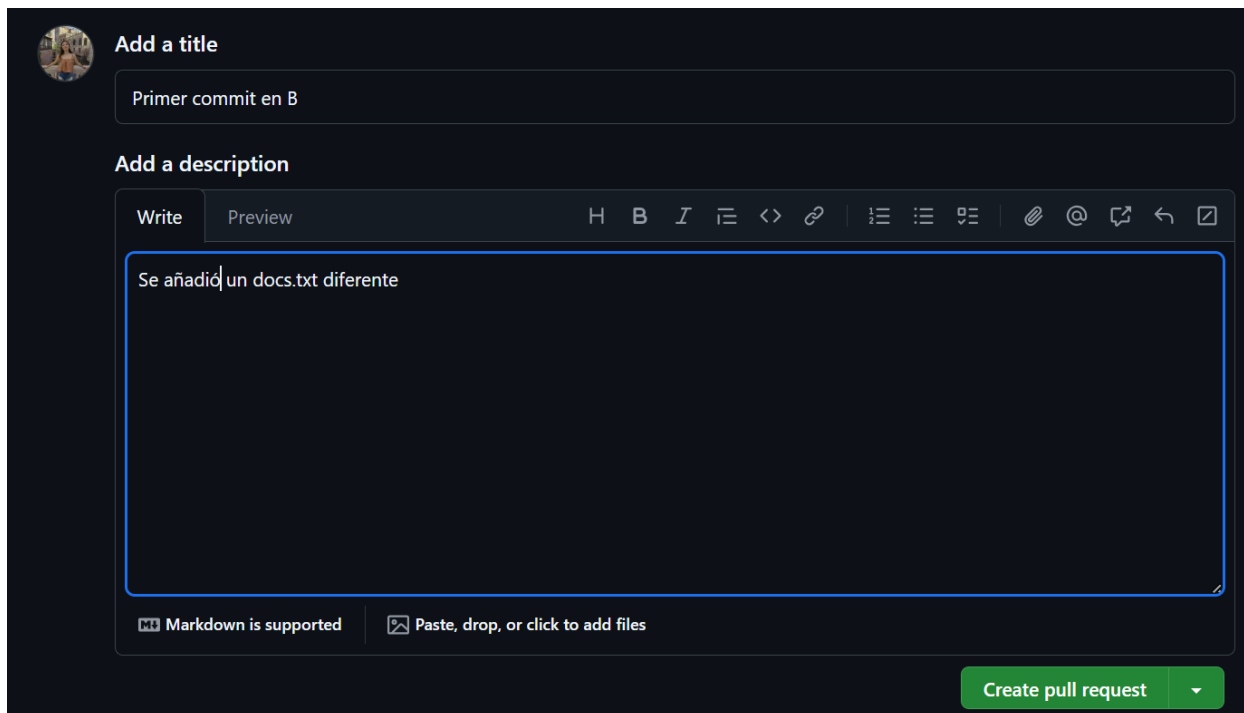
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git push -u origin B
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 301 bytes | 150.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'B' on GitHub by visiting:
remote:   https://github.com/valduque/TechnoReady/pull/new/B
remote:
To https://github.com/valduque/TechnoReady
 * [new branch]      B -> B
branch 'B' set up to track 'origin/B'.
```

4. PR created for branch "B":

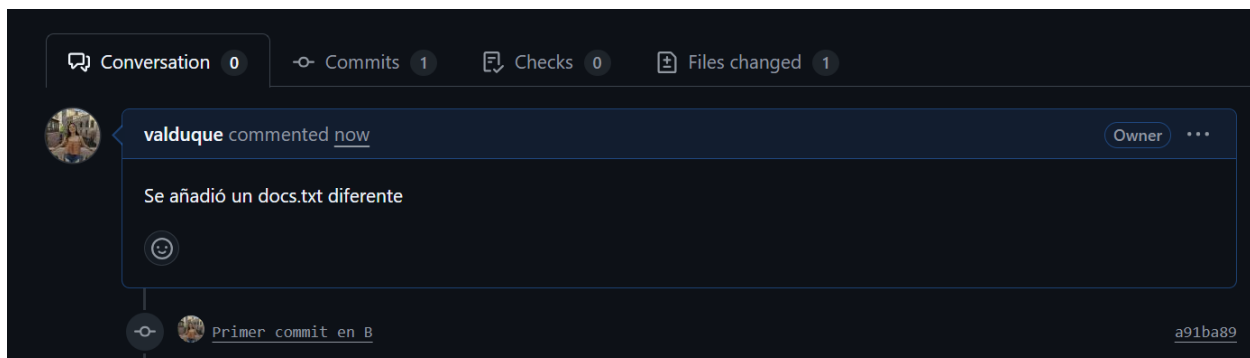
On the project's Github repository page, it will show the new branch and pushes created by it. "Compare and pull request" will be clicked:



Then, the customizable page for the pull request will appear:



Once finished all the details, just press "create pull request":



Conflict between branches:

When we want to merge both branches, the creation of docs.txt, will bring up problems. First, let's go to the main branch:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (B)
$ git checkout main
M      Challenge 1/Develop.docx
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

In main branch, let's merge with the A branch with main branch:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git merge A
Updating 16c1a45..5426a53
Fast-forward
 Challenge 1/~$evelop.docx | Bin 0 -> 162 bytes
 docs.txt                  | 1 +
 2 files changed, 1 insertion(+)
 create mode 100644 Challenge 1/~$evelop.docx
 create mode 100644 docs.txt
```

Then, merge the B branch with the main, to get the conflict:

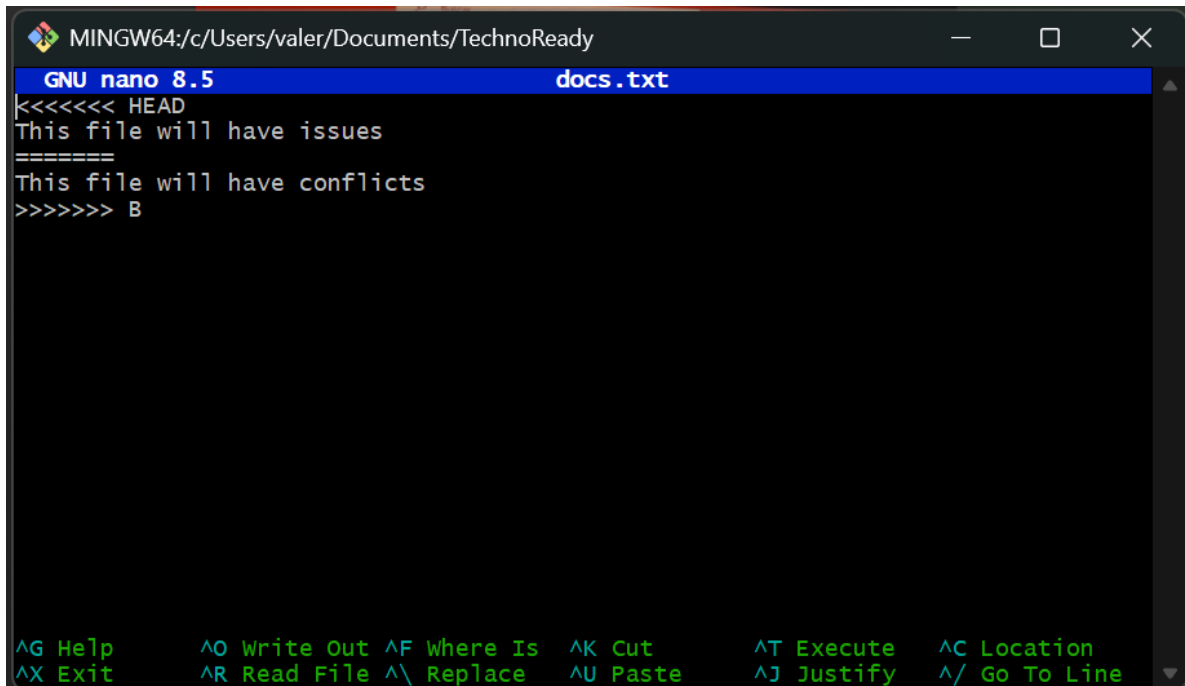
```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git merge B
Auto-merging docs.txt
CONFLICT (add/add): Merge conflict in docs.txt
Automatic merge failed; fix conflicts and then commit the result.
```

When this message It's showed, that means we are having a conflict between branches:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main)
$ git merge B
Auto-merging docs.txt
CONFLICT (add/add): Merge conflict in docs.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Conflict resolution process:

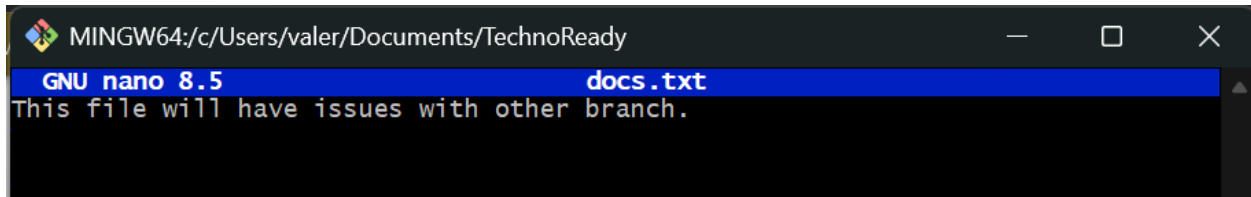
We face a challenge with the docs.txt, since now it has conflict;



```
MINGW64:/c/Users/valer/Documents/TechnoReady
GNU nano 8.5 docs.txt
|<<<<<< HEAD
This file will have issues
=====
This file will have conflicts
>>>>>> B

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

For this exercise, it was chosen to make the changes manually.



```
MINGW64:/c/Users/valer/Documents/TechnoReady
GNU nano 8.5 docs.txt
This file will have issues with other branch.
```

Git command used to revert changes:

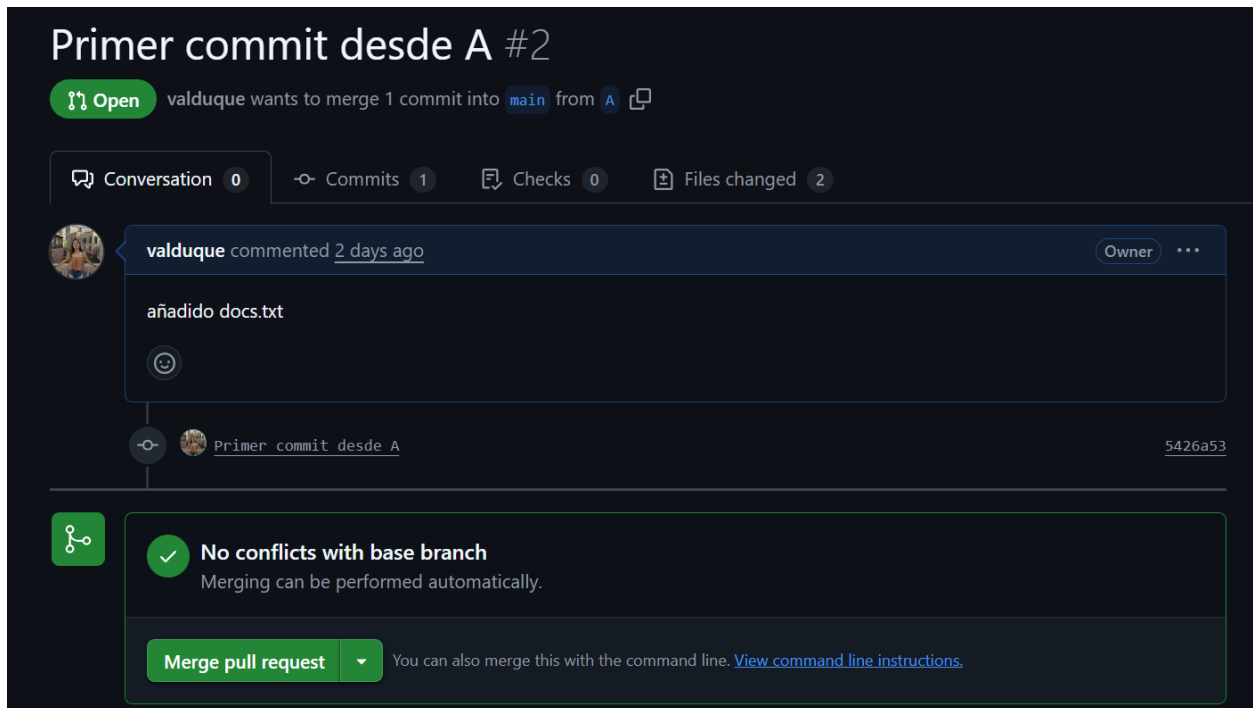
Once the changes are made. We finish the merge:

```
valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main|MERGING)
$ git add docs.txt

valer@LAPTOP-SI5J5VLM MINGW64 ~/Documents/TechnoReady (main|MERGING)
$ git commit -m "Docs.txt arreglado"
[main 8015b0e] Docs.txt arreglado
```

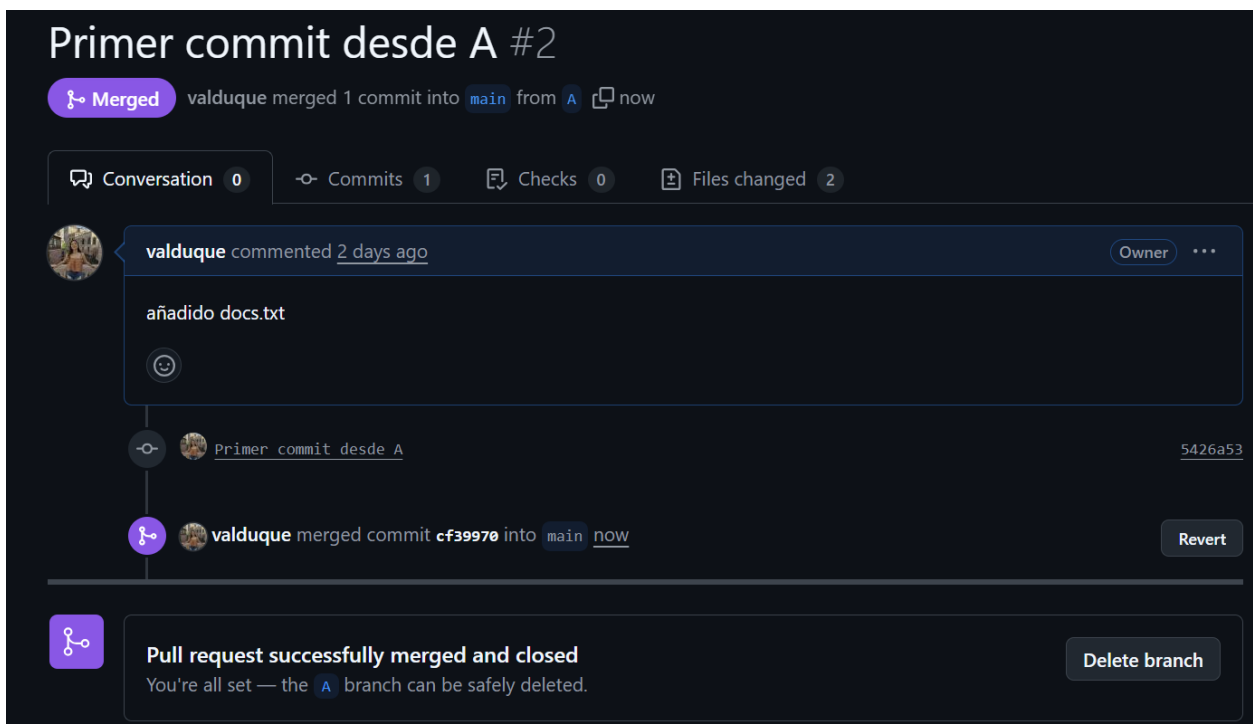
Approved PRs and merged branches:

Now, we can approve the pull request for both branches, in GitHub:



The screenshot shows a GitHub pull request interface. At the top, the title is "Primer commit desde A #2". Below the title, a green "Open" button is visible, followed by the text "valduque wants to merge 1 commit into main from A". The interface includes tabs for "Conversation" (0), "Commits" (1), "Checks" (0), and "Files changed" (2). A comment from "valduque" is shown, stating "añadido docs.txt". Below the comment, a commit link "Primer commit desde A" is visible. At the bottom, a green box indicates "No conflicts with base branch" and "Merging can be performed automatically." A green "Merge pull request" button is present, along with a link to "View command line instructions."

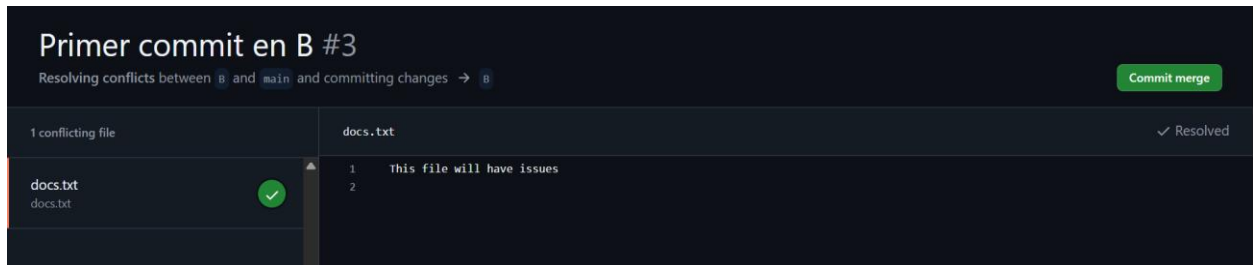
And everything is correct and working fine:



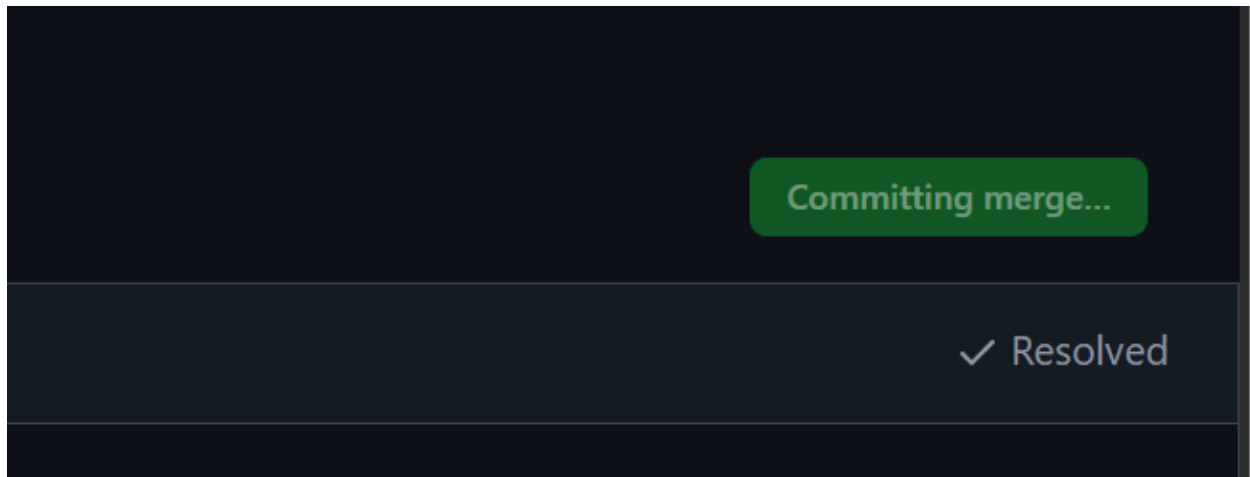
The screenshot shows the same GitHub pull request interface, but now the status is "Merged". The "Open" button has been replaced by a purple "Merged" button, and the text now says "valduque merged 1 commit into main from A now". The "Conversation" tab still shows the comment "añadido docs.txt". Below the comment, the commit link "Primer commit desde A" is visible. At the bottom, a purple box indicates "Pull request successfully merged and closed" and "You're all set — the A branch can be safely deleted." A "Delete branch" button is present. The "Merge pull request" button has been replaced by a "Revert" button.

5. Pull request the other branch:

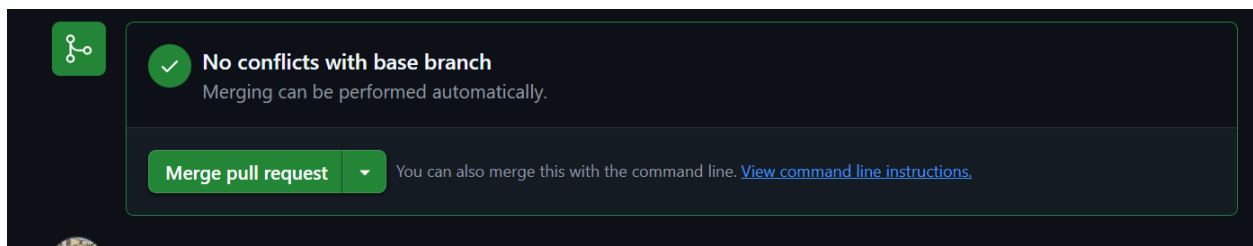
The other branch may or may not require you to fix the conflicted file. If you have to solve it just do it manually again on GitHub:



Then commit the merge:



And if there is not any issue, merge again:



Notes for feedback:

If there is only a branch in the local project, the first time it must be pushed with “git push -u origin name.branch”:

origin → your remote on GitHub.

A → the name of your local branch.

-u → links your local branch to the remote, so later you will only need to do git push.

There are plenty ways to resolve a conflict, it's easier to develop communication between collaborators in order to know which version must be followed.

It's not necessary the command “revert” if you already did manually changes.